Design and Implementation of a VLSI Chip for IoT Applications

Abhitha Nanda Kishore
SmartED Innovations

Abstract
This project involves the design and simulation of a VLSI chip for Internet of Things (IoT) applications, using Verilog for hardware description and simulation. The aim is to create an integrated circuit that meets the power efficiency, performance, and communication needs of IoT devices. Due to hardware limitations, the design is verified through online simulations, focusing on key aspects such as digital logic and power management. The report outlines the design process, from Verilog coding to simulation results, and discusses the potential applications of the VLSI chip in IoT systems.

Contents

1 Introduction

In today's interconnected world, the integration of **VLSI (Very Large Scale Integration)** technology with **IoT (Internet of Things)** applications is transforming industries by enabling compact, energy-efficient, and high-performance systems. VLSI technology plays a crucial role in the design of IoT-enabled devices, offering scalability and customization to meet diverse application requirements.

This project focuses on the design and simulation of a **motion detection** system integrated with an alert mechanism, tailored for IoT applications. The system employs VLSI principles to achieve a modular design consisting of:

1. A **motion detection** module that monitors state transitions and triggers alerts based on sensor inputs.
2. An **alert encoder** that generates encoded messages representing detection events.
3. A **transmitter** module that serially transmits encoded messages, simulating communication with a remote system.

The proposed system emphasizes functionality, reliability, and modularity, making it suitable for scenarios such as security systems, home automation, and surveillance. While power optimization strategies like clock gating have been explored as optional enhancements, the primary focus remains on a robust implementation of the core functionality.

This report details the methodology, implementation, and results of the design, highlighting the integration of IoT principles with VLSI technology to create a reliable and efficient motion detection system.

2 Objective

The objective of this project is to design and simulate a VLSI-based motion detection system integrated with an IoT-compatible alert mechanism. The system aims to achieve the following:

1. **Reliable Motion Detection:** Accurately identify motion based on state transitions between two sensors and trigger an alert for valid transitions.
2. **Efficient Data Encoding:** Encode motion detection events into a compact binary message, ensuring compatibility with IoT communication protocols.
3. **Seamless Data Transmission:** Transmit the encoded message serially to simulate communication with external devices or systems.
4. **Modular and Scalable Design:** Develop a modular architecture that facilitates scalability, ease of debugging, and potential enhancements like power-saving mechanisms.
5. **IoT Integration:** Create a system suitable for IoT applications such as security, automation, and surveillance, while adhering to VLSI principles for efficiency and performance.

3 Methodology

The methodology for designing and simulating the VLSI chip for IoT applications follows a structured approach, ensuring both high performance and power efficiency while addressing the specific needs of IoT systems. The tools, technologies, and step-by-step process employed in this project are outlined below.

1. **System Design and Architecture** The system is designed as a modular VLSI-based IoT application, implemented and simulated using Verilog on the **Kria K24C SOM board**. The design integrates three key components: **motion detection, alert encoding, and transmission**, ensuring seamless functionality while maintaining modularity. This architecture supports scalability for additional features or enhancements.

2. **Motion Detection** The motion detection module identifies motion using two sensors and state transitions. Valid transitions, such as $00 \rightarrow 01 \rightarrow 11$ or $11 \rightarrow 01 \rightarrow 00$, trigger motion alerts. Additionally, immediate detection is enabled when both sensors detect an object within a predefined proximity range (e.g., 40 cm). The logic prioritizes accuracy by focusing on intermediate states, minimizing false triggers.

3. **Alert Encoding** The detected motion event is encoded into a 16-bit binary message, formatted to include:

   - **Sensor ID (4 bits):** Identifies the module or source.
   - **Detection Flag (4 bits):** Indicates motion status.
   - **Reserved Data (8 bits):** Allows flexibility for future enhancements (e.g., range or timestamp data). This compact and structured encoding ensures compatibility with IoT communication protocols.

4. **Transmission Mechanism**

   The encoded message is transmitted serially via the tx out signal. A shift register mechanism ensures efficient bit-by-bit transmission starting with the most significant bit (MSB). The transmitter includes a completion flag (tx done) to signal the end of transmission, allowing the system to reset for the next detection.

5. **Platform Implementation** The system was implemented and simulated on the Kria K24C SOM board, leveraging its high-performance processing capabilities to validate the functionality of the design. The modular Verilog implementation ensures compatibility with the board's architecture, enabling seamless integration and scalability.

6. **Simulation and Testing** Extensive simulation was performed using Vivado to validate the functionality of each module and the integrated system. Key parameters evaluated include:

   - **Motion Detection Accuracy:** Verification of valid transitions and immediate detection.
   - **Encoding Integrity:** Ensuring the message format aligns with the requirements.
   - **Transmission Reliability:** Analyzing serial output (tx out) waveforms to confirm proper message transmission.

4 Code and Implementation Details

1. **Motion Detection Module**

```
module motion_detection(
    input wire sensor_a,       // Sensor A input
    input wire sensor_b,       // Sensor B input
    input wire clk,            // Clock signal
    input wire [7:0] range_a,  // Range from Sensor A
    input wire [7:0] range_b,  // Range from Sensor B
    output reg motion_detected // Motion detected output
);
    // State variables to track previous and current states
    reg [1:0] prev_state;
    reg [1:0] current_state;

    // Initialization
    initial begin
        prev_state = 2'b00;
        current_state = 2'b00;
        motion_detected = 0;
    end

    // Motion detection logic
    always @(posedge clk) begin
        // Update states based on sensor inputs
        prev_state <= current_state;
        current_state <= {sensor_b, sensor_a}; // Combine Sensor B and A inputs

        // Check for immediate detection based on range
        if (range_a <= 8'd40 && range_b <= 8'd40) begin
            motion_detected <= 1;
        end
        // Check valid state transitions for motion detection
        else if ((prev_state == 2'b00 && current_state == 2'b01) || // 00 → 01
                 (prev_state == 2'b01 && current_state == 2'b11) || // 01 → 11
                 (prev_state == 2'b11 && current_state == 2'b10) || // 11 → 10
                 (prev_state == 2'b10 && current_state == 2'b00))   // 10 → 00
        begin
            motion_detected <= 1;
        end else begin
            motion_detected <= 0; // Reset if no valid transition
        end
    end
endmodule
```

Functionality: This module tracks state transitions between `sensor_a` and `sensor_b` to detect motion. It also triggers an alert when both sensors detect objects within a 40 cm range.

2. **Alert Encoder**

```
    module alert_encoder(
    input wire motion_detected,     // Motion detection signal
    input wire clk,                 // Clock signal
    output reg [15:0] encoded_msg   // Encoded alert message
);
    // Always block to generate encoded message
    always @(posedge clk) begin
        if (motion_detected) begin
            // Example encoding format:
            // [15:12] = Sensor ID (e.g., 4'b0001)
            // [11:8] = Detection flag (e.g., 4'b1111 for motion detected)
            // [7:0] = Reserved space (currently set to 4'b0001)
            encoded_msg <= 16'b0001_1111_0000_0001;
        end else begin
            // Reset the message if no motion is detected
            encoded_msg <= 16'b0000_0000_0000_0000;
        end
    end
endmodule
```

Functionality: This module generates a 16-bit encoded message when motion is detected. The message format includes fields for sensor ID, detection flag, and reserved space for future enhancements.

3. **Transmitter Module**

```
    module transmitter(
    input wire clk,                 // Clock signal
    input wire [15:0] encoded_msg,  // Encoded message input
    input wire start_transmission,  // Signal to start transmission
    output reg tx_out,              // Serial transmission output
    output reg tx_done              // Transmission complete flag
);
    reg [15:0] shift_reg;           // Shift register for serial transmission
    reg [3:0] bit_counter;          // Counter for bits transmitted

    // Initialization
    initial begin
        shift_reg = 16'b0;
        bit_counter = 0;
        tx_out = 1;                 // Idle state
        tx_done = 0;
    end
```

```
    // Transmission logic
    always @(posedge clk) begin
        if (start_transmission) begin
            // Load the encoded message into the shift register
            shift_reg <= encoded_msg;
            bit_counter <= 16;      // Set counter to 16 bits
            tx_done <= 0;           // Clear transmission complete flag
        end else if (bit_counter > 0) begin
            // Transmit the MSB and shift left
            tx_out <= shift_reg[15];
            shift_reg <= shift_reg << 1;
            bit_counter <= bit_counter - 1;

            // Set tx_done when all bits are transmitted
            if (bit_counter == 1) begin
                tx_done <= 1;
                tx_out <= 1;         // Return to idle state
            end
        end else begin
            tx_done <= 0;           // Remain idle if no transmission
        end
    end
endmodule
```

Functionality: The transmitter module serially transmits the 16-bit encoded message using a shift register. The `tx_out` signal outputs one bit per clock cycle, starting with the MSB. The `tx_done` flag indicates the end of transmission.

4. **Top Module**

```
    module motion_detector_system(
    input wire sensor_a,            // Sensor A input
    input wire sensor_b,            // Sensor B input
    input wire clk,                 // Clock signal
    input wire [7:0] range_a,       // Range of Sensor A
    input wire [7:0] range_b,       // Range of Sensor B
    output wire motion_detected,    // Motion detected output
    output wire tx_out              // Transmitted alert message
);
    wire [15:0] encoded_msg;        // Encoded message

    // Instantiate motion detection module
    motion_detection motion_det (
        .sensor_a(sensor_a),
        .sensor_b(sensor_b),
        .clk(clk),
        .range_a(range_a),
        .range_b(range_b),
```

```
        .motion_detected(motion_detected)
    );

    // Instantiate alert encoding module
    alert_encoder encoder (
        .motion_detected(motion_detected),
        .clk(clk),
        .encoded_msg(encoded_msg)
    );

    // Instantiate transmitter module
    transmitter tx (
        .clk(clk),
        .encoded_msg(encoded_msg),
        .start_transmission(motion_detected),
        .tx_out(tx_out)
    );
endmodule
```

Functionality: The top module integrates the motion detection, alert encoder, and transmitter modules. It detects motion, encodes the event into a message, and transmits the message serially.

5. **Testbench**

```
module tb_motion_detector_system;
 reg sensor_a;         // Sensor A input
 reg sensor_b;         // Sensor B input
 reg clk;              // Clock signal
 reg [7:0] range_a;    // Range of Sensor A
 reg [7:0] range_b;    // Range of Sensor B
 wire motion_detected; // Motion detected output
 wire tx_out;          // Transmitted alert message

 // Instantiate the system
 motion_detector_system uut (
     .sensor_a(sensor_a),
     .sensor_b(sensor_b),
     .clk(clk),
     .range_a(range_a),
     .range_b(range_b),
     .motion_detected(motion_detected),
     .tx_out(tx_out)
 );

 // Clock generation
 initial begin
     clk = 0;
```

```
        forever #5 clk = ~clk; // 100 MHz clock
    end

    // Test stimulus
    initial begin
        // Initialize inputs
        sensor_a = 0;
        sensor_b = 0;
        range_a = 8'd50; // Default: No object in range
        range_b = 8'd50; // Default: No object in range

        // Case 1: Immediate detection and transmission
        #10 range_a = 8'd30; range_b = 8'd30; // Object detected within 40 cm
        #20 range_a = 8'd50; range_b = 8'd50; // Reset range

        // Case 2: State transition with transmission
        #10 sensor_a = 1;          // Sensor A detects object
        #50 sensor_b = 1;          // Sensor B detects object after delay
        #100 sensor_a = 0;         // Reset Sensor A
        sensor_b = 0;              // Reset Sensor B

        #200 $stop; // End simulation
    end

    // Monitor outputs
    initial begin
        $monitor("Time=%0t | Sensor A=%b | Sensor B=%b | Range A=%d  |
                Range B=%d | Motion Detected=%b | Tx Out=%b",
                 $time, sensor_a, sensor_b, range_a, range_b,
                 motion_detected, tx_out);
    end
endmodule
```

Functionality:

The testbench simulates the functionality of the motion detection system to verify its performance under various scenarios. A 100 MHz clock signal drives the modules, ensuring proper timing for motion detection, encoding, and transmission. The test cases include immediate motion detection, where both sensors detect an object within a predefined range (40 cm), and state-based detection, where transitions like 00 → 01 → 11 and 11 → 01 → 00 are simulated.

Key signals such as motion_detected, encoded_msg, and tx_out are monitored to ensure accurate functionality. For instance, when valid transitions occur, the motion_detected signal is triggered, and the encoded 16-bit message is transmitted serially via the tx_out output. The simulation results are

analyzed using waveforms in Vivado to confirm that the system behaves as expected in all scenarios. By testing both individual module behavior and system integration, this testbench validates the reliability and correctness of the design.

5 Results and Observations

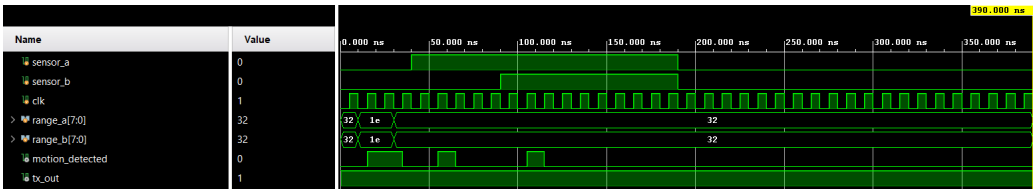

Figure 1: Setup of the Motion Detection System

In the textbffirst simulation, the system was tested for motion detection and basic functionality. The inputs, such as sensor signals and range values, were initialized, and scenarios like valid state transitions ($00 \rightarrow 01 \rightarrow 11$ and $11 \rightarrow 01 \rightarrow 00$) and immediate detection (both sensors within a 40 cm range) were simulated. The primary focus was on observing the motion detected signal, which toggled high during valid conditions. However, while the motion detection logic worked correctly, the **encoded message** (encoded msg) and **transmitter output** (tx out) were not explicitly monitored. As a result, their behavior could not be validated, and it was unclear if the alert message was being generated and transmitted correctly.

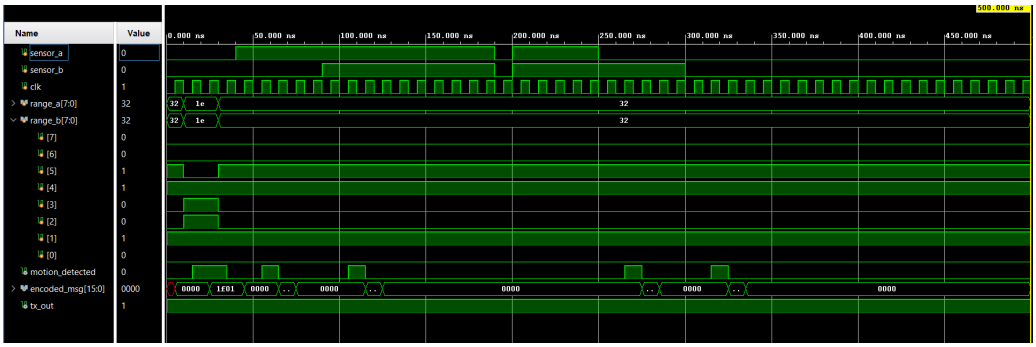

Figure 2: Transmission of encoded message

In the second simulation, the testbench was updated to explicitly monitor the encoded msg and its serial transmission via tx out. When motion was detected, the encoder generated a 16-bit binary alert message, and the transmitter sequentially transmitted the bits starting from the most significant bit (MSB). The waveforms confirmed the proper functioning of all modules:

1. The (motion detected) signal toggled high for valid state transitions and immediate detection.
2. The (encoded msg) displayed the expected alert message when motion was detected. (tx out) signal showed the serialized transmission of the encoded message, with each bit transmitted in sequence.

The second simulation provided a complete validation of the system, demonstrating its ability to detect motion, encode events into an alert message, and transmit the message reliably.

6 Conclusions

This project successfully demonstrates the design and simulation of a **VLSI-based motion detection system** integrated with an **IoT-compatible alert mechanism.** The modular design approach ensured seamless integration of key components: **motion detection, alert encoding, and transmission**. The system reliably detects motion based on state transitions between two sensors and immediate proximity detection, encodes the detection event into a structured binary message, and serially transmits the message to simulate communication with external devices.

The simulation results validated the system's functionality:

- Accurate motion detection for valid state transitions ($00 \rightarrow 01 \rightarrow 11$ or $11 \rightarrow 01 \rightarrow 00$) and immediate detection within a predefined range (40 cm).
- Proper encoding of detection events into a compact 16-bit binary message.
- Efficient serial transmission of the encoded message.

While the primary focus was on functionality, the project also explored optional enhancements such as power optimization through clock gating. Although this feature was not integrated into the final design, it remains a viable future improvement to enhance energy efficiency.

The project highlights the potential of VLSI technology in developing compact, efficient, and scalable IoT systems, making it suitable for applications such as security, automation, and surveillance. Future work could involve hardware implementation, advanced power-saving mechanisms, and real-time IoT network integration.

References

"IoT based Safety System for Women," 2021 6th International Conference on Communication and Electronics Systems (ICCES), Coimbatre, India, 2021, pp. 731-736

"Smart Women Safety Device Using IoT and GPS Tracker," 2023 Intelligent Computing and Control for Engineering and Business Systems (ICCEBS), Chennai, India, 2023, pp. 1-6